Episode 3: Writing Your First Dockerfile

Theory Notes - plain language, with a hands-on lab

The big idea: A **Dockerfile** is a plain text file with step-by-step instructions. Docker reads it and builds an **image**. You write the recipe once, and anyone can build the same image again and again.

1. Why do we need a Dockerfile?

In Episode 2 you ran ready-made images such as nginx. To package **your own app**, you must describe how to build its image: which base to start from, which files to add, what to install, and how to start it. The Dockerfile holds those steps in a file you can store in Git and review like code.

Story link: the Dockerfile is Riya's Recipe Card, `docker build` is the packing machine, and the image is the sealed kit.

2. The tiny project we will pack

Create a folder called `hello-flask` with three files.

app.py

```
from flask import Flask

app = Flask(__name__)

@app.route("/")
def home():
    return "Hello from Docker!"

if __name__ == "__main__":
    app.run(host="0.0.0.0", port=5000)
```

requirements.txt

```
flask==3.0.3
```

Dockerfile (no extension, capital D)

```
FROM python:3.11-slim
WORKDIR /app
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt
COPY . .
EXPOSE 5000
CMD ["python", "app.py"]
```

Very important: the app uses `host="0.0.0.0"`. Flask's default is 127.0.0.1, which means "only inside this container". Your browser would never reach it through the published port.

3. The instructions, one by one

Instruction	What it does	Story analogy
`FROM`	Chooses the **base image** to start from. Every Dockerfile begins with it. `python:3.11-slim` means Python 3.11 in a smaller (slim) Debian-based image.	Ready-made base dough
`WORKDIR`	Sets the working folder inside the image. It is created if it does not exist. Later COPY, RUN and CMD use it.	The workbench
`COPY`	Copies files from your computer (the build context) into the image. `COPY . .` means: everything here, into the current WORKDIR.	Bring ingredients to the bench
`RUN`	Runs a command **while building** and saves the result as a layer. Used for installing packages.	Prep at the factory
`EXPOSE`	Documents which port the app listens on. It does **not** publish the port. You still need `-p`.	A sign saying "we serve at window 5000"
`CMD`	The **default command** when a container starts. Can be replaced when you run the container. Only the last CMD counts.	What happens when the box opens
`ENTRYPOINT`	The fixed program that always runs. Extra arguments from docker run are added to it.	A machine fixed in the box
`ENV`	Sets an environment variable (we cover config in Episode 6).	A label on the box

4. Building the image

```
$ docker build -t hello-flask:1.0 .
```

`docker build` sends the **build context** (the folder given by the final dot) to Docker, then runs your Dockerfile top to bottom.

Part	Meaning
`-t hello-flask:1.0`	Tag: name **hello-flask**, version **1.0**. Without a tag you get the name latest.
`.`	The build context: the current folder. Docker can only COPY files that are inside it.
`-f FILE`	Use a Dockerfile with another name or location (optional).

Then check and run it:

```
$ docker images                # you will see hello-flask 1.0
$ docker run -d -p 8080:5000 --name hello hello-flask:1.0
$ curl http://localhost:8080    # or open it in a browser
Hello from Docker!
$ docker logs hello
$ docker stop hello && docker rm hello
```

Why 8080:5000? The app listens on **5000 inside** the container. We publish it on host port 8080 (HOST:CONTAINER). On macOS, host port 5000 is often taken by the AirPlay Receiver, so 8080 avoids a clash.

5. RUN vs CMD: build time vs run time

	RUN	CMD
When	While **building** the image	When a **container starts**
How often	Once per build (result stored in a layer)	Every time a container starts
Typical use	Install packages, create folders	Start the application
Example	`RUN pip install -r requirements.txt`	`CMD ["python", "app.py"]`

6. Layers and the build cache

Instructions such as RUN, COPY and ADD each create a **read-only layer**. Docker remembers layers it has built before. On the next build, if an instruction and the files it uses are unchanged, Docker reuses the layer (shown as CACHED). Once one layer changes, **all layers after it are rebuilt**.

That is why we copy `requirements.txt` first, install, and only then copy the rest of the code:

Bad order (slow)	Good order (fast)
COPY . . RUN pip install -r requirements.txt *Any code change reinstalls all libraries.*	COPY requirements.txt . RUN pip install -r requirements.txt COPY . . *Code changes reuse the installed libraries.*

> **Rule of thumb:** put what changes **least** near the top and what changes **most** (your code) near the bottom. See your layers with `docker history hello-flask:1.0`.

7. CMD vs ENTRYPOINT

Dockerfile	docker run IMG	docker run IMG other args
`CMD ["python","app.py"]`	python app.py	**Replaces** CMD: runs the other args
`ENTRYPOINT ["python"]`	python	**Appends**: python other args
`ENTRYPOINT ["python"]` `CMD ["app.py"]`	python app.py	Replaces CMD only: python other args

Exec form vs shell form: `CMD ["python","app.py"]` (JSON array, double quotes) is the **exec form** and is preferred: the app runs directly as the main process and receives stop signals properly. `CMD python app.py` is the **shell form**: it runs through `/bin/sh -c`. Single quotes in the array break the JSON and Docker falls back to shell form.

To override an ENTRYPOINT at run time use `--entrypoint`.

8. COPY vs ADD

COPY only copies files. **ADD** can also download URLs and auto-extract local tar files, which can surprise you. Use **COPY** unless you truly need ADD.

9. Rebuilding after a change

A running container never changes when you edit the Dockerfile or code. The steps are: **edit, rebuild the image, replace the container**.

```
$ docker build -t hello-flask:1.1 .
$ docker stop hello && docker rm hello
$ docker run -d -p 8080:5000 --name hello hello-flask:1.1
```

10. Lab checklist

- Create the three files and build with a tag.
- Run `docker build` a second time and notice CACHED lines.
- Change the text in app.py, rebuild, and see only the last layers rebuild.
- Override CMD: `docker run --rm hello-flask:1.0 python --version`.
- Look at `docker history hello-flask:1.0`.

11. Cheat sheet

Goal	Command or instruction
Build an image	docker build -t NAME:TAG .
List images	docker images
See layers	docker history NAME:TAG
Add a second tag	docker tag NAME:1.0 NAME:stable
Delete an image	docker rmi NAME:TAG
Base image	FROM image:tag
Install at build time	RUN command
Default start command	CMD ["exe","arg"]

12. Key takeaways

- Dockerfile = recipe, image = built kit, container = running copy.
- RUN happens at build time, CMD at run time.
- Order matters: dependencies first, code last, for fast rebuilds.
- EXPOSE documents; -p publishes. Bind the app to 0.0.0.0.
- Changed something? Rebuild the image and start a new container.

13. Self-check quiz

Q1. What is the difference between RUN and CMD?
Q2. What does the dot at the end of docker build mean?
Q3. Why copy requirements.txt before the rest of the code?
Q4. Does EXPOSE 5000 make the app reachable from your browser?
Q5. What happens if a Dockerfile has two CMD lines?

Answers

A1. RUN executes while building and is saved in the image; CMD is the default command when a container starts.
A2. The build context: the folder whose files Docker can use.
A3. So the pip install layer stays cached when only code changes.
A4. No. It only documents the port. You need -p.
A5. Only the last CMD is used.

Homework: Add a second route to app.py (for example /about), rebuild as version 1.1, and compare the build time with the first build. **Next episode:** Volumes and networking.